LAB PRACTICE

P3. A word around a ring with MPI

Several copies of one program, a word that travels between them, and the first measurement of what running at the same time buys you.

What you should be able to do when you finish

- Compile and run a program with several processes, and explain what `mpicc` and `mpirun` each do.
- Say how a process knows which of the copies it is, and how many copies exist.
- Explain why a send needs a matching receive, and what happens when it does not find one.
- Measure the difference between work done one after another and work done at once.

What you are given

File	What it is	State
P3_ring.c	Process 0 reads a word and sends it around a ring of processes. Each one prints that it has the word.	Contains three defects

What is wrong with it

Three defects of three kinds: one the compiler rejects, one that appears the first time you run it, and one that appears only with more than two processes. Correct them one at a time.

Building and running

```
# the laboratory machines already have everything you need.
# on your own computer, if mpicc is missing and you have
# administrator rights:  sudo apt install build-essential libopenmpi-dev

# MPI programs are built with mpicc, not with gcc
mpicc -Wall -o P3_ring P3_ring.c

# run four copies of the program; process 0 waits for you to type a word
mpirun -np 4 ./P3_ring

# if your machine has fewer cores than processes you ask for
mpirun --oversubscribe -np 8 ./P3_ring

# give the word without typing it, so that a measured time does not include you
time (echo hello | mpirun --oversubscribe -np 8 ./P3_ring)
```

Before you change anything

Build it once, unmodified, and take a screenshot of the first error. You will hand in a screenshot of each of the three, so capture each one at the moment it appears, with the command that produced it visible.

Hand in five separate files, not compressed

P3_ring.c, P3_ring_after.c, P3_ring_before.c, results.txt and evidence_NAME_SURNAME_ddmmyy.pdf. Use exactly these names. If you only complete stage one, hand in the three of them that stage one asks for. The table at the end of the next page says what goes inside each file.

A note on how you work

You may use any tool to understand the code. You may not hand in anything you cannot explain, and you can be asked in class to justify any line.

THE WORK

Three stages

Stage one is compulsory. Stages two and three separate the marks.

STAGE 1	Make the word travel
Start from	P3_ring.c as you received it.
What to do	Correct the three defects. Above each correction write a comment saying what was wrong and how you noticed it. Keep the corrected file as P3_ring.c: every later stage starts from it.
What to observe	With four processes the word passes through 1, 2 and 3 and returns to 0, which reports that the ring is closed.
What to hand in	P3_ring.c corrected; the three screenshots, in the evidence PDF; the output of one successful run with four processes, in results.txt.

STAGE 2	Where the time goes
Start from	The corrected P3_ring.c.
What to do	Make two copies. In P3_ring_after.c add `sleep(2);` just after a process receives the word. In P3_ring_before.c add the same `sleep(2);` before the ring starts, so all processes wait at once. Both need `#include <unistd.h>`.
What to measure	Run each of the two with eight processes, giving the word with echo as shown on page 1, and keep the line real printed by `time`.
What to hand in	P3_ring_after.c and P3_ring_before.c; in results.txt, the two real times and two sentences: why one total is close to the sum of the waits and the other to a single wait.

STAGE 3	Break it on purpose
Start from	The corrected P3_ring.c.
What to do	Three runs: one process; more processes than cores without the oversubscribe option; the same with it. `nproc` prints the number of cores.
What to observe	What is printed in each case, and whether the program ends.
What to hand in	In results.txt, for each run: the command, what you saw, and one sentence explaining it.

What you hand in: five files, exact names

File	Stage	What it contains
P3_ring.c	1	The corrected program, compiling without warnings with `-Wall`, with your comment above each of the three corrections.
P3_ring_after.c	2	The corrected program with `sleep(2);` after receiving the word.
P3_ring_before.c	2	The corrected program with `sleep(2);` before the ring starts.
results.txt	1 to 3	Plain text copied from your terminal, in three parts headed exactly `STAGE 1`, `STAGE 2` and `STAGE 3`. Stage 1: the output of the run with four processes. Stage 2: the two real times and the two sentences. Stage 3: the three runs.
evidence_NAME_SURNAME_ddmmyy.pdf	1 to 3	One PDF. First the three screenshots, in the order the errors appeared, each with one line saying which defect it shows. Then the handwritten page, scanned, with your name and the date: one short paragraph for each of what failed and how you knew, what you changed, what you measured.

How it is marked

Level	What it looks like
Not enough	It does not compile, or the screenshots are missing.
Sufficient	The three defects are corrected and the word travels.
Good	The above, plus the two times of stage two, correctly read.
Very good	The above, plus the three runs of stage three explained.